

- **DCL—Data Control Language:** These statements control access to the data. There are only two of them: *GRANT* and *REVOKE*.
- **TCL—Transaction Control Language:** Transactions are handled using these statements. Examples: *COMMIT*, *ROLLBACK*.

Designing a database

Here's where we design a database for some simple data. Let's say you want to make a database to store the names, addresses, phone numbers and birthdays of your friends and relatives. The first step is to see what kind of data you want to store:

- Names
- Addresses
- Phone Numbers
- Birthdays

Let's begin by creating the database in MySQL:

```
mysql> CREATE DATABASE mycontacts;
Query OK, 1 row affected (0.00 sec)
```

```
mysql> USE mycontacts;
Database changed
```

The *USE* command tells MySQL which database you want to be working on in this session. You can list the available databases using the following command:

```
mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mycontacts |
| mysql |
| phpmyadmin |
+-----+
4 rows in set (0.05 sec)
```

Now let's go table by table, and see what columns to put into each. These are the columns I put in:

Names:

- contact_id
- name

Addresses:

- contact_id
- address_line1
- address_line2
- city
- country
- email

Phone Numbers:

- contact_id
- landline

- mobile
- Birthdays:
- contact_id
- birthdate

So, we can now make four tables: *NAMES_MASTER*, *ADDRESS_MASTER*, *PHONE_MASTER* and *BIRTHDAY_MASTER*. The MySQL command for making *NAMES_MASTER* would be:

```
mysql> CREATE TABLE NAMES_MASTER (
    contact_id INT,
    name VARCHAR(50)
);
```

```
Query OK, 0 rows affected (0.14 sec)
```

Other tables can be made similarly. *VARCHAR* and *INT* are data-types. *INT* stands for integer. *VARCHAR* is a variable character. It's a dynamically changing field, whose size depends on the entered data. The *VARCHAR(50)* means the field can have a maximum size of fifty characters. Other supported data-types are listed in the MySQL documentation.

Now let's explore the concept of primary and secondary keys. If you notice, the field *contact_id* is common to all tables. This is to relate the tables by that common field. Here, the *contact_id* in *NAMES_MASTER* is the primary key, and the rest are secondary keys. In other words, *contact_id* is the reference for all the data in each table (as the secondary key), where it finally relates to the name of the contact (as primary key).

Once a table is made, you can populate it using a tool called phpMyAdmin. Doing justice to using phpMyAdmin would require another article; hence, I won't cover this aspect here. A link to this is given at the end of this article. You can also populate using the *INSERT* statement:

```
INSERT into table_name (column1, column2....)
values {value1, value2...};
```

At the end of this exercise, you will have a database ready to use. This, frankly, is just the starting point in designing a database. I hope it proved helpful to you in understanding what goes on beneath the surface in cyberspace, at least when it comes to managing data. So, till next time, cheerio folks! 

Links

- [1] MySQL Home—<http://www.mysql.com>
- [2] Data, Information, Knowledge and Wisdom <http://www.systems-thinking.org/dikw/dikw.htm>
- [3] phpMyAdmin—<http://www.phpmyadmin.net/>

By: Siddharth Mankad

The author is a new media designer, with an affinity for physical computing, computer software and code, and country music.